

Technical Requirements Document

Take Your Trip (tytluxe.in) — Travel Agency Booking Platform

Version: 1.0 (Draft) **Owner:** [Your name], Software Architect **Companion doc:** Take-Your-Trip-PRD.md v1.0 **Status:** Draft for review **Last updated:** July 25, 2026

0. Scope Note — Reconciling with the PRD

The PRD scoped v1 as an **enquiry/lead-generation MVP** and explicitly deferred live GDS integration and online payments to a later phase. The stack you've specified (TripJack for live Hotel/Flight inventory, Razorpay for payments) is booking-engine-grade, not just lead-gen-grade. This TRD assumes the **real intent is to build the booking engine now**, and treats it as a phased rollout on one architecture rather than a rewrite later:

- **Phase 1 (build first):** Architecture, DB, auth, and TripJack integration are built to support live search + booking end-to-end, but initially only **enabled for Hotels** (TripJack's simpler, lower-risk vertical). Flights, Cruises, Staycations, Packages launch in enquiry-only mode using the *same* listing/enquiry tables — no throwaway code.
- **Phase 2:** Enable TripJack Flights booking + Razorpay for Hotels/Flights once Phase 1 is stable in production.
- **Phase 3:** Cruises/Staycations/Packages move from enquiry-only to bookable as supplier contracts allow (these typically stay manually curated with agent-set pricing, not TripJack).

This phasing is a recommendation, not a hard requirement — flag it for stakeholder sign-off before development starts.

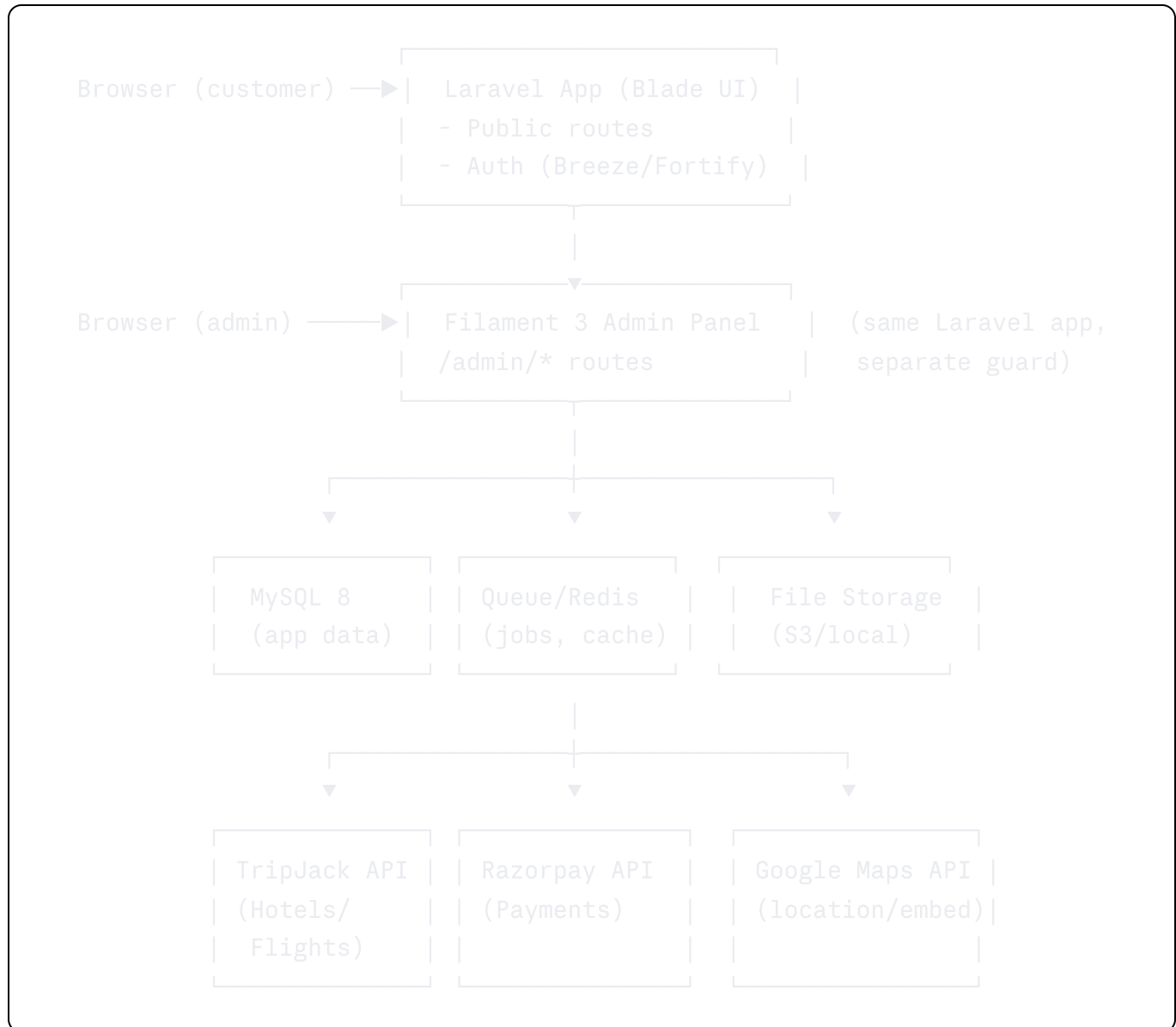
1. Architecture Overview

1.1 Style

Monolithic Laravel application (modular monolith), server-rendered with Blade + Bootstrap 5 + vanilla JS, using Laravel as both the customer-facing site and the API layer for TripJack/Razorpay server-to-server calls. Filament 3 runs in the same codebase as the admin panel.

Why monolith over microservices: Team size, MVP timeline, and Laravel's maturity all favor one deployable unit. Splitting into services (search service, booking service, payment service) adds operational overhead (multiple deploys, service discovery, distributed transactions for booking+payment) with no current benefit at this traffic scale. Laravel's module boundaries (see 1.3) give code separation without deployment complexity. Revisit only if a specific module (e.g., search) needs independent scaling.

1.2 High-Level Diagram (textual)



1.3 Internal Module Boundaries (within the monolith)

Organize Laravel by domain module (not just default `app/Http`), to keep the "modular" part of "modular monolith" real:

```

app/
  Modules/
    Hotels/      (models, services, TripJack hotel client, controllers)
    Flights/     (models, services, TripJack flight client, controllers)
    Cruises/     (models, services – curated/manual, no TripJack)
    Staycations/ (models, services – curated/manual)
    Packages/    (models, services – curated/manual)
    Enquiries/   (shared enquiry/lead capture used by all verticals)
    Bookings/    (shared booking + payment orchestration)
    Offers/      (promo/offer management)
  Support/
    TripJack/    (API client, DTOs, response mappers – vertical-agnostic)
    Razorpay/    (payment client, webhook handlers)
    GoogleMaps/  (geocoding/embed helpers)

```

Each supplier integration lives in `Support/`, consumed by module services — so if TripJack's contract changes or a second supplier is added later, only `Support/TripJack` and its adapter interface change, not every controller that uses it.

2. Frontend Stack

Layer	Choice	Notes
Markup	HTML5 + Blade templating	Blade for layouts, components, partials (<code>@include</code> , <code><x-component></code>)
Styling	Bootstrap 5	Existing site is Elementor/WordPress; new build should extract the current visual design (colors, typography, spacing) into a Bootstrap 5 theme/SCSS override rather than default Bootstrap look, to preserve brand identity
Interactivity	Vanilla JavaScript	No SPA framework. Use <code>fetch()</code> for AJAX (search filters, enquiry form submission, seat/room selection widgets)
Asset pipeline	Laravel Vite	Bundles/minifies CSS+JS; use <code>@vite</code> directive in Blade layouts
Icons	Bootstrap Icons or existing site's icon set	Match current brand

Why no frontend framework (React/Vue): The site is content- and form-driven, not app-like/highly stateful. Blade + Bootstrap + vanilla JS keeps one language (PHP) and one

deploy target for the whole team, matches the specified stack, and avoids the API-first overhead a SPA would require (every page would need a JSON API even for simple content). If a specific page later needs rich interactivity (e.g., a multi-step flight seat map), scope vanilla JS or Alpine.js for that component only — don't introduce a framework for the whole app.

Key frontend components to build:

- Destination/date/pax search widgets (Hotels, Flights) with autocomplete (backed by TripJack's static/lookup data, cached)
 - Listing grid with filters (price range, category, star rating) — server-side filtering via query params + Blade, progressively enhanced with JS for instant filter feel
 - Booking/enquiry multi-step forms
 - Razorpay Checkout.js integration (client-side payment modal)
 - Google Maps embed on hotel detail pages
-

3. Backend Stack

Layer	Choice	Notes
Language/runtime	PHP 8.2	Enables enums, readonly properties, fibers — use enums for booking status, payment status, vertical type
Framework	Laravel 12	Routing, ORM, queues, jobs, events, validation, Blade
Admin panel	Filament 3	Built on Livewire; runs as a panel within the same Laravel app at <code>/admin</code>
Queue driver	Redis (or database driver if Redis unavailable initially)	Async jobs: TripJack booking confirmation polling, email/WhatsApp notifications, Razorpay webhook processing
Cache	Redis	Cache TripJack static/lookup data (city lists, airport codes), search results (short TTL), Filament performance
Session	Database or Redis	Avoid file-based sessions once on more than one app server
Background jobs	Laravel Queues + Horizon (if Redis)	TripJack booking is not always synchronous; queue confirmation/status checks
Scheduled tasks	Laravel Scheduler (cron)	Expire offers past <code>valid_to</code> , sync TripJack static data nightly, reconcile pending payments

Why Laravel 12 fits this project: Built-in auth scaffolding, Eloquent ORM speeds up the Hotels/Flights/Cruises/Packages data modeling, queues are essential for reliable third-party API orchestration (TripJack booking + Razorpay webhooks are not instant/synchronous by nature), and Filament gives a production-grade admin panel without hand-building CRUD screens — directly satisfying the PRD's "admin can manage listings without a developer" requirement.

4. Database

4.1 Engine

MySQL 8.0, InnoDB engine (transactional integrity needed for bookings/payments).

4.2 Core Schema (representative, not exhaustive)

users		
id, name, email, phone, password, role (customer/agent/admin), email_verified_at, timestamps		
hotels	flights_cache (optional)	cruises
id	(TripJack live search results	id
tripjack_hotel_id (nullable)	are not persisted long-term –	title,
description		
title, description	cache only for session/	
cruise_line, itinerary (json)		
destination_id (FK)	comparison purposes)	
duration_nights		
category (enum: beach/city/...)		category
(enum)		
address, lat, lng		
price_from		
price_from, star_rating		images
(json)		
is_curated (bool)		
is_active		
source (enum: tripjack/manual)		
is_active		
staycations	packages	offers
id	id	id
title, description	title, description	title,
description		
price_from, duration	inclusions (json: flight/	
discount_type/value		
amenities (json)	hotel/activities)	
applies_to (vertical + optional FK)		
is_active	price_from	
valid_from, valid_to		
	is_active	
is_active		
destinations	enquiries	bookings
id, name, country, type	id	id
(used across Hotels/Flights/	user_id (nullable – guest ok)	user_id
Packages for search/filter)	vertical (enum: hotel/flight/	
vertical (enum)		
	cruise/staycation/package)	

```

reference_id (poly: hotel/flight/etc.)
tripjack_booking_id (nullable)
(enum: pending/confirmed/
cancelled/failed)
amount, currency
traveler_details (json)
created_at, updated_at

reference_id (nullable, poly)
name, phone, email
status
travel_date_from/to
pax_adults, pax_children
notes
status (new/contacted/
quoted/converted/closed)
assigned_agent_id (nullable)
payments
source (web/whatsapp/phone)
id,
booking_id (FK)
razorpay_order_id
razorpay_payment_id
amount, status
raw_response (json)

```

Design notes:

- `enquiries` is shared across all five verticals via a `vertical` enum + nullable polymorphic reference — matches the PRD's requirement that every vertical supports "Request a Quote," and avoids five near-duplicate tables.
- `bookings` and `payments` are separate tables: a booking can exist before payment completes (TripJack hold), and a payment can be retried/reattempted without duplicating the booking record.
- `source` (tripjack vs manual) on `hotels` lets Hotels support both live TripJack inventory and hand-curated "featured" listings in the same table/UI, matching the brand's "curated" positioning even on a live-inventory vertical.
- Flights are intentionally **not** stored as a persistent catalog (unlike hotels) — flight search results are live/volatile by nature; only confirmed bookings are persisted in `bookings`.

- JSON columns (itinerary, amenities, inclusions, traveler_details) used for semi-structured data that varies by vertical, avoiding excessive join tables for an MVP-stage product.

4.3 Indexing & Performance

- Index (destination_id), (category), (is_active) on hotels/staycations for filter queries
- Index (status), (vertical), (created_at) on enquiries and bookings for admin dashboard queries
- Full-text index (MySQL (FULLTEXT)) on hotel/package titles+descriptions for search-as-you-type

5. Authentication

Concern	Approach
Customer auth	Laravel Breeze (Blade edition) — email/password + optional social login (Google) later; session-based, not token-based, since frontend is server-rendered
Guest checkout/enquiry	Enquiries and Hotel bookings should not require login — PRD targets "low-digital-confidence travelers"; forcing signup kills conversion. Capture name/phone/email inline and optionally offer "create an account to track this" post-submission
Admin/agent auth	Separate Filament guard ((admin) guard, own (admins) table or (role) flag with route middleware) — keeps admin auth fully isolated from customer auth for security
Password policy	Laravel default hashing (bcrypt/argon2), min 8 chars, Laravel's built-in breach/rate-limit protections on login
2FA	Recommended for (admin)/(agent) roles via Filament's built-in 2FA (or Laravel Fortify) — admin panel touches customer PII and payment data
API auth (future, if a mobile app is added)	Laravel Sanctum, deferred — not needed for a Blade-only v1

6. APIs & Third-Party Integrations

6.1 TripJack (Hotels & Flights)

- **Integration pattern:** Server-side only. Laravel backend calls TripJack REST API using an HTTP client (Guzzle via Laravel's (Http) facade); credentials/API keys stored in

`.env`, never exposed to frontend.

- **Search flow:** User submits search form → Laravel calls TripJack search endpoint synchronously (with a short timeout, e.g., 8-10s) → results mapped to a common DTO → rendered in Blade. Cache identical searches for a short TTL (e.g., 2-5 min) to reduce API calls and cost.
- **Booking flow:** TripJack "block/hold" → Razorpay payment → TripJack "confirm booking" on payment success → store in `bookings` with `tripjack_booking_id`. This must be a queued/idempotent job, not purely synchronous — network failures between hold and confirm must not silently lose a paid booking.
- **Failure handling:** If TripJack confirm fails after successful payment, auto-flag the booking `status = failed_needs_review` and alert an agent (email/Slack) — never leave a paid customer with an unconfirmed booking silently.
- **Rate limits/cost:** Cache static lookup data (cities, airports, hotel static content) nightly via scheduled job rather than calling on every page load.

6.2 Razorpay (Payments)

- **Integration pattern:** Razorpay Orders API (server-side create order) + Razorpay Checkout.js (client-side modal) + **server-side webhook** for payment confirmation — never trust the client-side success callback alone for marking a booking paid.
- **Webhook endpoint:** `POST /webhooks/razorpay`, signature-verified using Razorpay's webhook secret, idempotent (check `razorpay_payment_id` not already processed before updating booking status).
- **Refunds/cancellations:** Handled via Razorpay Refunds API, triggered by admin action in Filament, logged in `payments`.

6.3 Google Maps

- **Use case:** Embed map + geocode address on hotel detail pages (per PRD's hotel detail requirements).
- **Integration pattern:** Server-side Geocoding API call once at listing creation time (store lat/lng on `hotels`, don't re-geocode on every page view); client-side Maps Embed/JS API just for rendering the map, using a **restricted** API key (HTTP referrer restriction to the production domain).

6.4 Internal APIs

No public REST API is required for v1 (no mobile app, no third-party consumers). Internal AJAX endpoints (e.g., `/api/hotels/search`, `/api/enquiries`) are JSON-returning Laravel routes consumed only by the site's own vanilla JS — protect with CSRF tokens and rate limiting, not full API authentication.

7. Deployment Plan

7.1 Environments

- **Local:** Laravel Sail or Herd/Valet + local MySQL
- **Staging:** Mirrors production, used for TripJack/Razorpay **test/sandbox** credentials, QA before release
- **Production:** Live TripJack + Razorpay live keys

7.2 Hosting

Component	Recommendation
App server	VPS/cloud (e.g., DigitalOcean, AWS Lightsail/EC2, or managed Laravel hosting like Laravel Forge + a provider) — Nginx + PHP-FPM 8.2
Database	Managed MySQL 8 (e.g., AWS RDS, DigitalOcean Managed DB) — automated backups, easier to scale/restore than self-hosted
Queue worker	Supervisor-managed <code>(php artisan queue:work)</code> process(es), separate from web process
Redis	Managed Redis instance or same VPS for MVP scale
File storage	S3-compatible storage (AWS S3 or DO Spaces) for hotel/package images — not local disk, for durability and CDN-ability
CDN	Cloudflare (or similar) in front of the app — caches static assets, provides DDoS protection, easy SSL

7.3 CI/CD

- Git-based workflow (GitHub/GitLab), `(main)` = production, `(develop)`/feature branches for work
- CI pipeline: run `(phpunit)`/`(pest)` tests, `(composer)` audit, PHPStan/Larastan static analysis on PR
- Deployment: Laravel Forge, Envoyer, or a simple GitHub Actions → SSH deploy script running `(composer install --no-dev)`, `(php artisan migrate --force)`, `(php artisan config:cache)`, `(php artisan queue:restart)`
- Zero-downtime deploys recommended once traffic is meaningful (Envoyer or Forge's zero-downtime deploy)

7.4 Migration from current static site

1. Recreate current static content (About, brand assets, testimonials) as seeded Laravel/Blade content — don't attempt to migrate WordPress DB, rebuild is faster

given the small page count.

2. DNS cutover: point tytluxe.in to new app only after staging sign-off and TripJack/Razorpay live-mode testing is complete.
 3. Keep a maintenance-mode fallback page ready during cutover.
-

8. Security Requirements

Area	Requirement
Transport	HTTPS everywhere (HSTS enabled), TLS 1.2+ only
Secrets	All API keys (TripJack, Razorpay, Google Maps, DB credentials) in <code>.env</code> , never committed to git; use separate keys per environment
Payment data	Never store raw card data. Razorpay Checkout.js handles card entry (PCI scope stays with Razorpay); only store <code>razorpay_order_id</code> / <code>razorpay_payment_id</code> /status
Webhook verification	Razorpay webhook signature must be verified server-side before trusting payment status
Input validation	Laravel Form Requests for all form submissions (enquiry, booking, search) — server-side validation regardless of client-side JS validation
CSRF	Laravel's default CSRF protection on all state-changing routes/forms
XSS	Blade's automatic escaping (<code>{{ }}</code>) by default; audit any <code>{!! !!}</code> raw output (e.g., admin-entered rich text descriptions) — sanitize with a library like HTMLPurifier if Filament rich text editor is used for listing descriptions
SQL injection	Eloquent ORM/query builder parameterizes queries by default; no raw string-concatenated SQL
Admin panel access	IP allowlist optional but recommended for <code>/admin</code> ; mandatory strong password + 2FA for all admin/agent accounts; role-based access control within Filament (agents see enquiries/bookings only, not billing/settings)
Rate limiting	Laravel's built-in rate limiter on login, search, and enquiry-submission endpoints to prevent abuse/spam and control TripJack API cost exposure
PII handling	Customer name/phone/email/traveler details are PII — restrict DB access, avoid logging full payloads containing PII in plaintext application logs, define a data retention policy for old enquiries
Dependency hygiene	<code>composer audit</code> in CI; keep Laravel/Filament/PHP patched — travel/payment apps are attractive targets
Backups	Automated daily DB backups with tested restore process, given payment/booking data is business-critical

Area	Requirement
Logging/monitoring	Centralized error tracking (e.g., Sentry) for booking/payment failures specifically — these need fast human attention per Section 6.1's failure-handling requirement

9. Key Technical Decisions & Reasons (Summary)

Decision	Reason
Modular monolith over microservices	Matches team size/timeline; Laravel module boundaries give separation without deployment complexity
Blade + Bootstrap 5 + vanilla JS (no SPA)	Matches specified stack; content/form-driven site doesn't need SPA complexity; single-language team
Shared <code>enquiries</code> table across verticals (not 5 separate tables)	PRD requires enquiry capture on every vertical; avoids schema duplication, simplifies the admin dashboard
Flights not persisted as a catalog (only bookings are)	Flight inventory/pricing is inherently live/volatile via TripJack; persisting search results would go stale immediately
Separate <code>bookings</code> and <code>payments</code> tables	A TripJack hold can exist pre-payment; payment retries shouldn't duplicate bookings
Razorpay confirmation via server-side webhook, not client callback alone	Client-side callbacks can be spoofed or lost on network failure; webhook is the source of truth
Guest checkout/enquiry allowed (no forced login)	PRD explicitly targets "low-digital-confidence" travelers; login walls hurt conversion, which is the core PRD success metric
Phased TripJack/Razorpay rollout (Hotels first)	De-risks the most complex integration (real-time booking + payment) on one vertical before expanding, without building throwaway architecture
Filament for admin	Directly satisfies PRD's "non-technical staff can manage listings/offers" requirement with minimal custom build
Managed MySQL + S3-compatible storage over self-hosted	Payment/booking data is business-critical; managed backups and durable image storage reduce operational risk for a small team

10. Open Technical Questions for Stakeholder Input

- Does the business have a live TripJack contract/credentials yet, and is it sandbox-tested before go-live?

- Should Cruises/Staycations/Packages ever integrate a supplier API, or do they stay permanently agent-curated (affects whether to build a generic "supplier adapter" interface now vs. later)?
 - What's the expected initial traffic/booking volume — this affects whether a single VPS is sufficient at launch or managed/scalable infra should be provisioned from day one?
 - Is a mobile app anticipated within 12 months? (Affects whether to build Sanctum-based API auth now vs. later.)
-

This TRD should be read alongside the PRD's MVP scope (Section 6) and Section 8 (features to avoid) — where they conflict, as with TripJack/Razorpay being introduced ahead of the PRD's original enquiry-only MVP, flag for explicit re-scoping sign-off before development begins.